Name: Aksharaa.B

Reg No: 192311278

Q1. 0/1 Knapsack Optimization A warehouse manager must load items into a truck that has a fixed carrying capacity. Each item has a specific weight and associated profit, and items cannot be split or reused. The manager needs to decide which combination of items should be selected so that the total profit is maximized without exceeding the weight limit. Use Dynamic Programming to determine the optimal profit. Sample Input: n = 3 Values = 60 100 120 Weights = 10 20 30 W = 50 Expected Output: Maximum Profit = 220

Project Overview

The 0/1 Knapsack Problem is a classical optimization problem in Design and Analysis of Algorithms (DAA).

In this project, a warehouse manager has a truck with a limited carrying capacity. There are several items available, where each item has:

- A weight
- A profit/value
- A decision: either select the item completely (1) or do not select it (0)

An item cannot be partially selected and cannot be selected more than once.

Aim : Maximize the total profit while ensuring that the total weight does not exceed the truck's capacity.

Dynamic Programming is used to efficiently determine the optimal combination of items.

Understanding the 0/1 Concept

The name 0/1 Knapsack comes from the fact that every item has only two possible choices:

Number of items = 3

Item 1:	Item 2:	Item 3:
Weight = 10	Weight = 20	Weight = 30
Profit = 60	Profit = 100	Profit = 120

Algorithm: 0/1 Knapsack using Dynamic Programming

Input:

n = number of items

Values[1...n] = profit of each item

Weights[1...n] = weight of each item

W = maximum capacity of the truck

Output:

Maximum Profit

Step 1: Create a DP table DP[0...n][0...W]

Step 2: Initialize:

For w = 0 to W

DP[0][w] = 0

Step 3: For i = 1 to n

For w = 0 to W

If Weights[i] <= w then

DP[i][w] = max(

Values[i] + DP[i-1][w - Weights[i]],

DP[i-1][w]

)

Else

DP[i][w] = DP[i-1][w]

Step 4: Maximum Profit = DP[n][W]

Step 5: Print Maximum Profit

For the given input

n = 3

Values = [60, 100, 120]

Weights = [10, 20, 30]

W = 50

The optimal selection is:

- Item 2 → Weight = 20, Profit = 100
- Item 3 → Weight = 30, Profit = 120

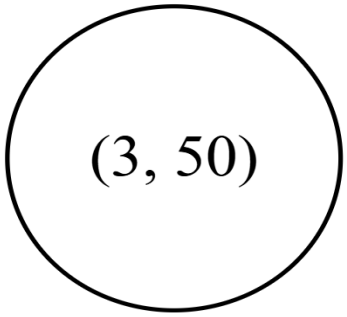
Total Weight = 50

Total Profit = 220

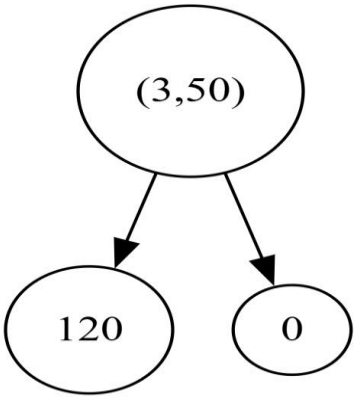
Maximum Profit = 220

Time Complexity: O(n × W)
Space Complexity: O(n × W)

Step1 :

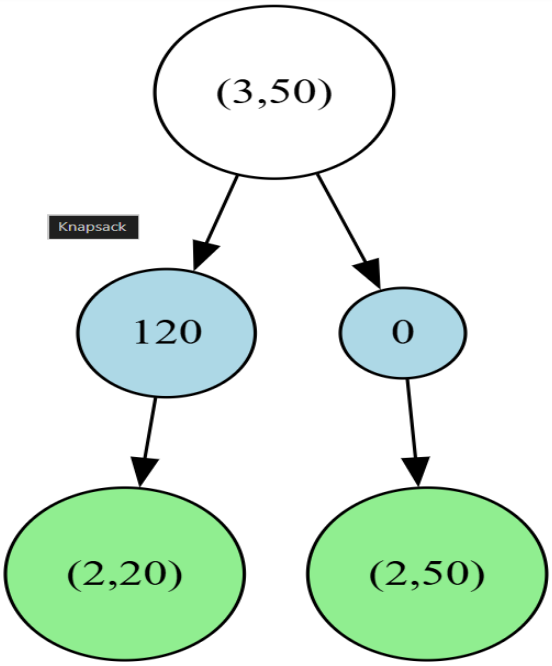


STEP1: (3,50) represents the initial state with **3 items available** and **50 units of capacity**.



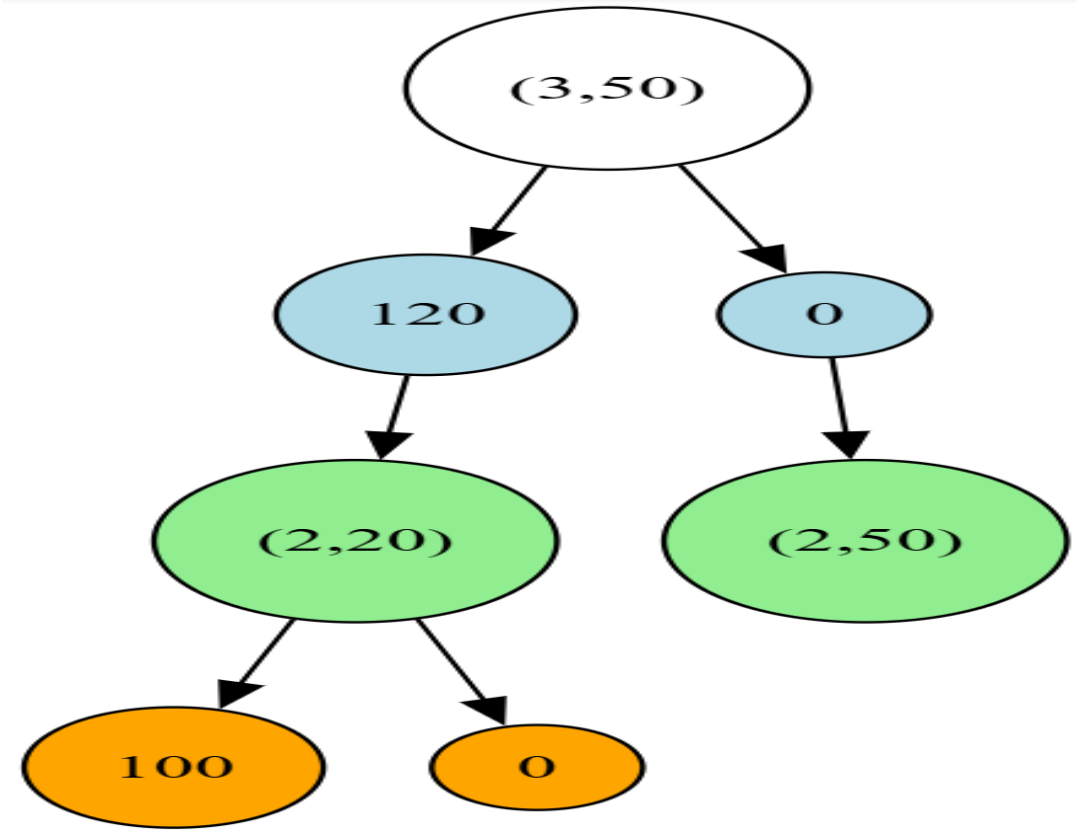
STEP 2 – Item 3 Selection

Item 3 has **weight 0 and profit 120**.



STEP 3 : Item 3 has **weight 30 and profit 120**. Two choices are considered:

- **Include** → profit 120, remaining capacity 20 → **(2,20)**



Step 4– Expand (2,20)

Item 2 has **weight 20 and profit 100**.

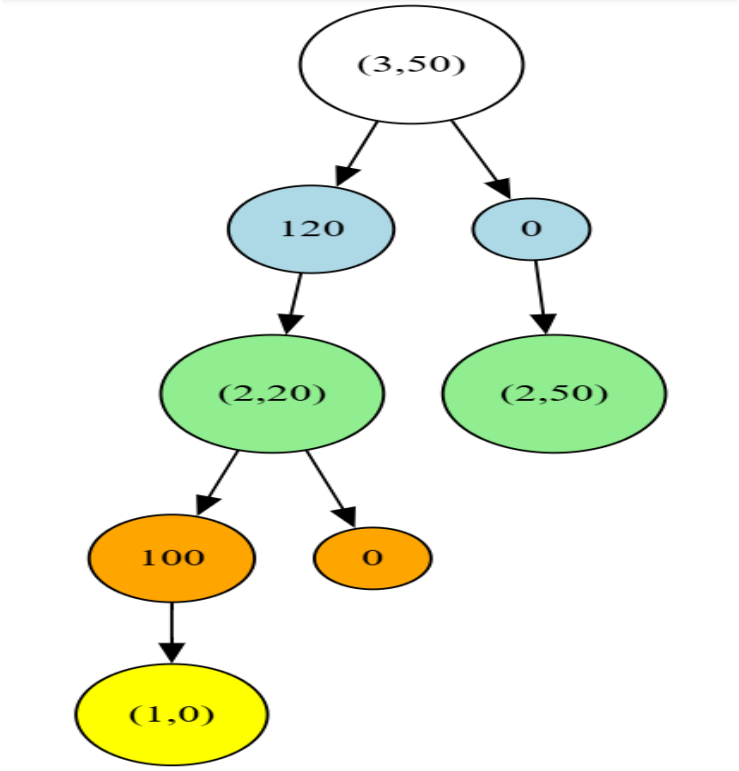
- **Include** → profit 100, remaining capacity 0 → **(1,0)**
- **Exclude** → profit 0, remaining capacity 20 → **(1,20)**
- Profit = 120
- Remaining capacity = 50 - 30 = 20
- Remaining items = 2

So: → **(2,20)**

Exclude Item 3:

- Profit = 0
- Capacity remains 50
- Remaining items = 2

So: (2,50)

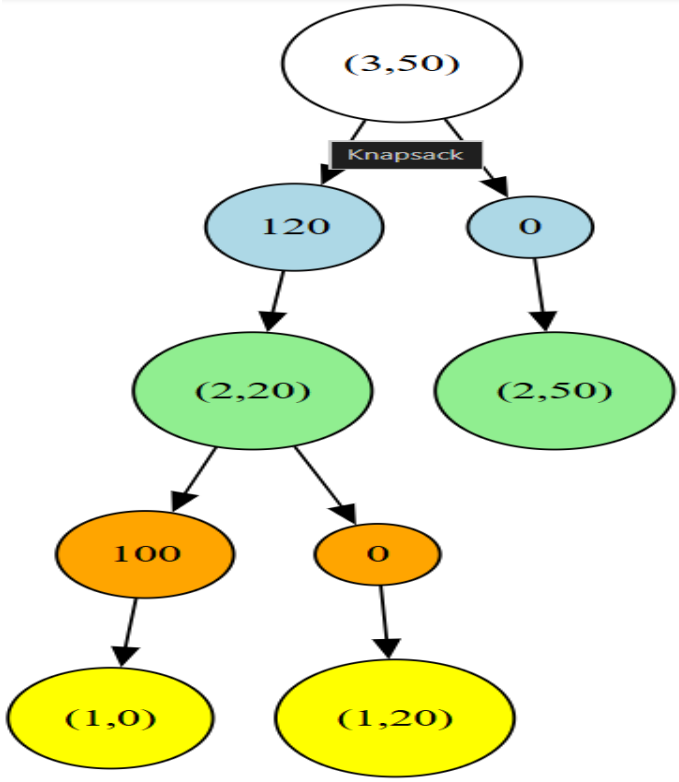


Include Item 2:

20 - 20 = 0

So: → **Profit = 100 → (1,0)**

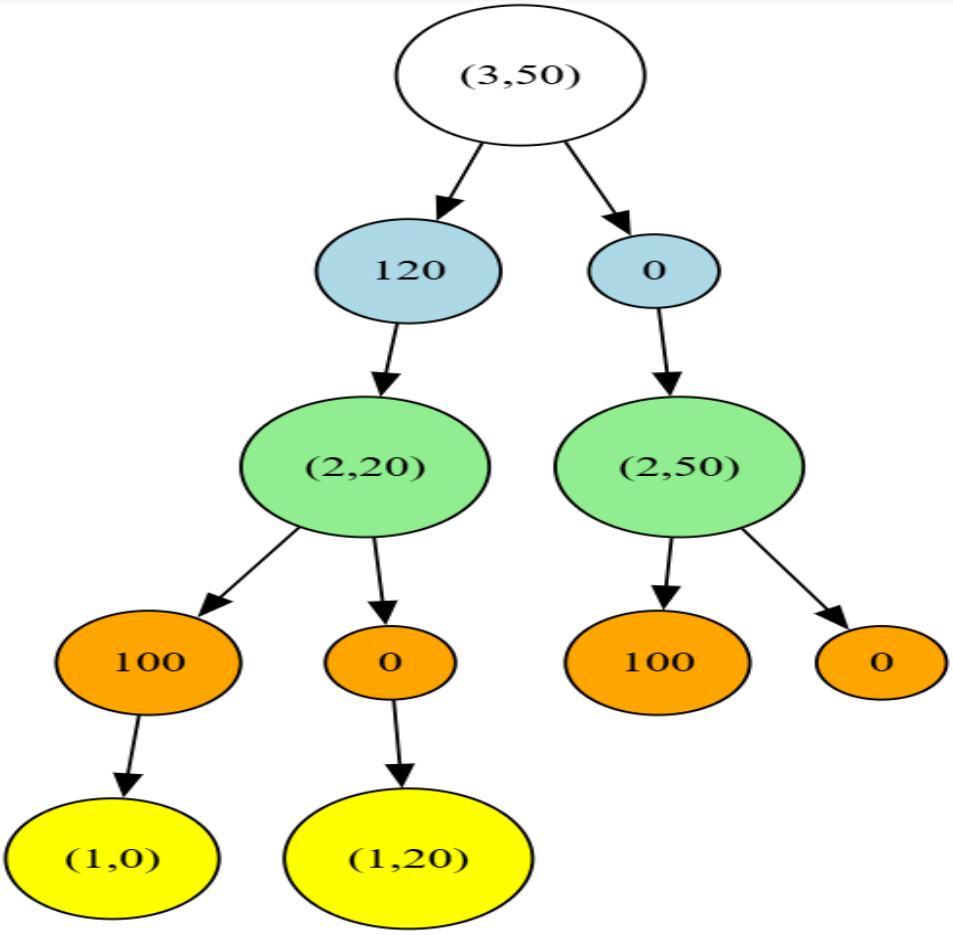
Since the remaining capacity is 0, no more items can be included.



Exclude Item 2:

Capacity stays 20.

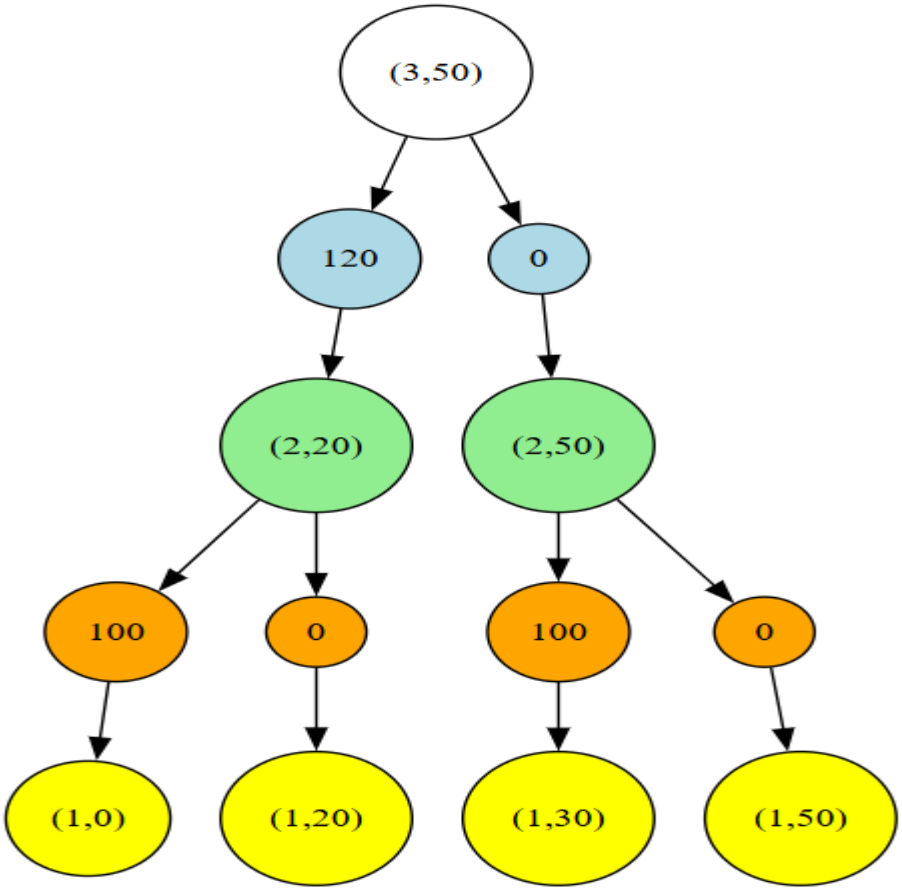
So: → **Profit = 0 → (1,20)**



Step 5 – Expand (2,50)

Item 2 is considered again.

- **Include** → profit 100, remaining capacity 30 → **(1,30)**
- **Exclude** → profit 0, remaining capacity 50 → **(1,50)**



Expand (2,50)

The right side is also important.

From (2,50), consider Item 2:

50 - 20 = 30

So we get:→ **(1,30) with profit 100**

If Item 2 is **excluded**:

Capacity remains 50.

So:

→ (1,50) with profit 0

Conclusion:

The 0/1 Knapsack problem was successfully solved using the Dynamic Programming approach. Each item was evaluated using two possible decisions: include the item or exclude the item. The state-space tree represents these decisions and shows the remaining number of items and available capacity at each stage.

For the given input:

- Number of items: 3
- Values: 60, 100, 120
- Weights: 10, 20, 30
- Knapsack capacity: 50

The optimal solution is to select Item 2 and Item 3.

- Total weight: 20 + 30 = 50
- Total profit: 100 + 120 = 220

Therefore, the maximum achievable profit is 220 without exceeding the truck's capacity. Dynamic Programming provides an efficient way to solve the problem by storing previously computed results and avoiding unnecessary repeated calculations.